

Cloud-Based Movie Streaming Platform: A Full-Stack Netflix-Inspired Web Application

Mustakim Ahmed Hasan (2421349042), MD. Nafiz Al Mamun (2232050642),
Tanim Ahmed (2312597042), Ahamed Osman Asif (2031080642)

CSE299.4; Group: 01

Department of Electrical & Computer Engineering
North South University

Abstract—This report presents the design, implementation, deployment, and evaluation of a cloud-based movie streaming platform developed as a full-stack CSE299 project. The system is inspired by modern subscription video platforms and includes user authentication, protected routing, movie browsing, detail pages, HLS-based playback, watch history, continue-watching behavior, typo-aware search, responsive user interface components, and cloud deployment. The frontend is implemented with Next.js and Tailwind CSS, while the backend is implemented with FastAPI, SQLAlchemy, asyncpg, and PostgreSQL hosted on Supabase. Media assets are delivered through public cloud storage URLs, and both the frontend and backend are deployed on Vercel. The project emphasizes practical software engineering principles such as separation of concerns, API-driven development, reusable components, environment-based configuration, defensive error handling, secure password storage, token-based authentication, CORS policy configuration, and production-oriented database connection handling. Special attention was given to real deployment problems such as cross-origin requests, serverless database pooling, prepared statement conflicts, auth state synchronization across domains, and frontend loading-state polish. The final result is a working end-to-end streaming application that demonstrates the architecture and user experience patterns of modern media platforms.

Index Terms—Movie streaming, Next.js, FastAPI, PostgreSQL, Supabase, HLS, Vercel, authentication, fuzzy search, cloud deployment.

I. INTRODUCTION

Cloud-based streaming platforms are now a common model for delivering entertainment content to users across devices. Services such as Netflix, Disney+, Apple TV, and other streaming platforms combine content catalogs, secure user accounts, personalized browsing, watch history, high-quality playback, and scalable deployment infrastructure. The goal of this project was to build a simplified but realistic version of such a platform using modern web development tools.

The project implements a full-stack web application where authenticated users can browse movies, view detailed information, search for titles, play video streams, and keep track of viewing history. The application is divided into a Next.js frontend and a FastAPI backend. The frontend is responsible for user interface rendering, route protection, search input handling, player interaction, loading states, and responsive design. The backend is responsible for authentication, movie APIs, history APIs, search ranking, database access, and

CORS configuration. PostgreSQL on Supabase stores users, movies, genres, subtitles, and watch history.

A major objective was to make the system work in both local and deployed environments. Deployment introduced real production concerns such as CORS configuration, serverless database pooling, SSL behavior, cross-domain authentication, and database connection handling.

II. PROJECT OBJECTIVES

The main objectives of the project are listed below:

- Build a full-stack movie streaming platform using a modern frontend and backend architecture.
- Implement user registration, login, logout, protected pages, and persistent auth state.
- Provide a movie browsing experience with homepage sections, movie cards, details pages, history pages, and search results.
- Implement HLS-based video streaming using browser video APIs and hls.js.
- Store and retrieve data using PostgreSQL through SQLAlchemy asynchronous sessions.
- Add typo-tolerant search that ranks related movies even when the exact query is not present.
- Deploy the frontend and backend to Vercel and connect them to Supabase.
- Apply practical software engineering principles such as modularity, reusable components, environment variables, structured API routes, and defensive error handling.

III. TECHNOLOGY STACK

Table I summarizes the major technologies used in the platform.

Next.js was selected for routing, layouts, server/client components, and production build support. FastAPI was selected for typed Python APIs, dependency injection, and clear route organization. PostgreSQL was used for relational data such as users, movies, genres, subtitles, and watch history. Supabase hosts the database and media resources, while Vercel hosts the frontend and backend deployments.

IV. SYSTEM ARCHITECTURE

The application follows a three-tier cloud architecture. The user interacts with the Next.js frontend in the browser. The

TABLE I
TECHNOLOGY STACK

Layer	Technology
Frontend	Next.js App Router, React, TypeScript, Tailwind CSS
Backend	FastAPI, Python, SQLAlchemy async ORM, asyncpg
Database	PostgreSQL hosted on Supabase
Storage	Supabase Storage/public media URLs
Streaming	HLS playlists (M3U8), hls.js, HTML5 video
Authentication	JWT, HTTP-only backend cookie, frontend token cookie, Bearer auth fallback
Deployment	Vercel frontend and Vercel FastAPI backend
Version Control	Git and GitHub

frontend sends API requests to the FastAPI backend. The backend performs validation, authentication, business logic, and database operations through SQLAlchemy and asyncpg. PostgreSQL stores structured application data, and Supabase-hosted media URLs provide video stream resources.

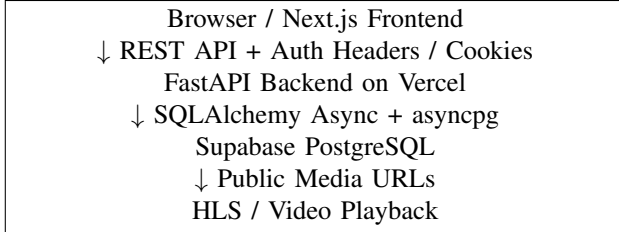


Fig. 1. High-level system architecture.

The frontend and backend are deployed as separate Vercel projects. This separation makes the architecture closer to real production systems where frontend and backend deployments can be scaled, configured, and updated independently. The backend exposes versioned API routes under `/api/v1`. The frontend uses an API helper to call those routes using an environment variable called `NEXT_PUBLIC_API_BASE`. This makes the application portable between local development and production.

V. FRONTEND DESIGN AND FEATURES

The frontend is implemented using Next.js with the App Router. It contains page routes under `frontend/app`, shared UI components under `frontend/components`, and reusable client/server helpers under `frontend/lib`. The UI was built with Tailwind CSS to allow rapid styling while keeping components readable and reusable.

A. Layout and Navigation

The global layout is implemented in `frontend/app/layout.tsx`. It renders the shared `SiteHeader`, route protection, and the page content. The navigation bar includes branding, auth actions, history access, and search. It was refined so that the search bar remains visible across normal

application pages and is hidden only on login, registration, and active player pages. This creates a more consistent browsing experience.

B. Homepage

The homepage is implemented in `frontend/app/page.tsx`. It displays a cinematic hero section, movie rows, cards, search results when a query is active, and a *Continue Watching* section based on watch history. The hero preview uses video playback for the featured content, while movie rows allow the user to browse available titles.

Important homepage features include:

- featured hero section;
- movie card rows;
- search-aware homepage state;
- result count for active searches;
- continue-watching row from history data;
- responsive spacing and loading skeletons.

C. Movie Cards and Rows

Movie cards are implemented in `frontend/components/MovieCard.tsx`, while horizontal sections are implemented in `frontend/components/Row.tsx`. Cards show movie posters, title metadata, hover behavior, focus states, and navigation links. UI polish was added through transitions, shadows, scale effects, and keyboard-friendly focus rings.

D. More Information Pages

Movie details are shown on pages such as `frontend/app/more-info/[id]/page.tsx`. These pages provide more descriptive metadata about a movie, including title, year, maturity rating, poster/backdrop information, and play/navigation actions. This separates browsing from playback so users can inspect content before starting a stream.

E. History Page

The history page is implemented in `frontend/app/history/page.tsx`. It shows movies the user has watched or partially watched. Empty states were improved so the page no longer appears broken when there is no history. Instead, the user sees a clean panel explaining that no watch activity has been recorded yet.

F. Loading Skeletons and Error States

A shared skeleton component file, `frontend/components/Skeletons.tsx`, provides consistent loading UI across routes. The application uses shimmer-style placeholders instead of abrupt blank screens. Error states were also improved to show user-friendly panels rather than raw text. This provides a more professional experience and helps users understand whether data is loading, unavailable, or empty.

VI. BACKEND API DESIGN

The backend is implemented using FastAPI and organized around route modules. The entry point is `backend/app/main.py`, which creates the FastAPI app, configures CORS, registers routers, and defines a health endpoint. API modules are separated into authentication, movies, and history.

A. API Route Structure

The backend uses versioned routing under `/api/v1`. The main route files are:

- `backend/app/api/routes/auth.py` for authentication;
- `backend/app/api/routes/movies.py` for movie listing, details, search, trending, and recommendations;
- `backend/app/api/routes/history.py` for watch history retrieval and updates;
- `backend/app/api/deps.py` for shared authentication dependencies.

FastAPI dependency injection is used to provide database sessions and authenticated user objects to route handlers. This keeps route code cleaner and centralizes repeated logic.

B. Movie API

The movie API supports listing movies, retrieving individual movie details, retrieving trending movies, retrieving latest releases, and generating movie recommendations. Movie responses include metadata such as title, description, year, duration, rating, poster URL, backdrop URL, HLS master URL, genres, and subtitles.

The backend also hydrates related data such as genres and subtitles. Defensive error handling is used so that if optional related data cannot be retrieved, the movie response can still be returned with empty related fields. This prevents one missing optional relationship from breaking the whole browsing page.

C. History API

The history API stores watch progress per user and movie. When the player reaches certain playback milestones, it posts progress to the backend. The backend either updates an existing watch history row or creates a new one. The history list is sorted by most recently updated items. This supports both the History page and the Continue Watching row on the homepage.

D. Recommendation Support

The recommendation service in `backend/app/services/recommendations.py` provides simple content-related retrieval. It can recommend movies based on genres and user history. Although this is not a full machine-learning recommendation engine, it demonstrates the data flow required for personalized rows.

TABLE II
DATABASE TABLES AND PURPOSE

Table	Purpose
users	Stores email, full name, hashed password, and account creation time.
movies	Stores movie metadata, poster/backdrop URLs, HLS stream URL, and creation time.
genres	Stores unique genre names.
movie_genres	Many-to-many association between movies and genres.
subtitles	Stores subtitle language, label, and URL per movie.
watch_history	Stores user-specific movie progress and completion status.

VII. DATABASE DESIGN

The database uses PostgreSQL with SQLAlchemy ORM models. The main tables are listed in Table II.

SQLAlchemy async sessions are configured in `backend/app/db/session.py`, following the asynchronous engine/session model supported by SQLAlchemy. The backend normalizes PostgreSQL URLs to the `postgresql+asyncpg` dialect, removes incompatible query parameters, configures SSL behavior, and switches to a serverless-safe database pool configuration when running on Vercel.

A. Supabase PostgreSQL

Supabase was used as the PostgreSQL hosting provider, and its documentation recommends pooler modes depending on whether the client is persistent or serverless. Local development can use a direct database URL, while production serverless deployment uses a transaction pooler URL. This distinction is important because serverless functions create short-lived connections and can scale quickly. Connection pooling prevents the application from exhausting database connection limits.

B. Serverless Connection Handling

A major production issue was caused by prepared statement caching when using `asyncpg` with the Supabase transaction pooler. Transaction pooling can reuse database connections in a way that conflicts with client-side prepared statement names. The fix was to set `statement_cache_size = 0` in serverless environments. This disables `asyncpg` prepared statement caching for Vercel while keeping the application stable with the pooler.

VIII. AUTHENTICATION AND AUTHORIZATION

Authentication includes registration, login, logout, current-user lookup, route protection, and navbar state synchronization.

A. Registration and Login

Users register with email, full name, and password. The backend checks whether the email already exists, hashes the password, and stores the user. During login, the backend verifies the password and generates a JWT access token. The token contains the user's email as the subject and includes an expiration time.

B. Password Hashing

Password storage uses bcrypt, a common password hashing library for server applications. Before bcrypt hashing, the password is normalized using SHA-256. This avoids bcrypt input length limitations while preserving deterministic verification. The security module also includes a fallback path for legacy bcrypt verification.

C. JWT and Session Handling

The backend creates signed JWTs using `python-jose`, following the standard JWT encode/decode model and expiration-claim behavior. The token is set as an HTTP-only cookie by the backend. Because the deployed frontend and backend are hosted on different domains, the frontend also stores a frontend-domain token cookie and sends the token as a Bearer token when needed. The backend authentication dependency accepts the backend cookie first and falls back to the `Authorization: Bearer` header when the cookie is unavailable.

D. Protected Routes

The frontend uses a route guard component to prevent logged-out users from accessing protected pages. Public routes include login and registration. Protected pages require a successful `/auth/me` check. If the user is not authenticated, the frontend redirects to login.

E. Auth State and Navbar Stability

The navbar originally suffered from a loading flicker where login/signup buttons could appear briefly even for logged-in users. This was solved by server-seeding the initial auth state and using shared auth-fetching utilities. The navbar now receives initial user data from a server helper and revalidates through a shared client helper. This reduces flicker and keeps authenticated navigation stable.

IX. CACHING AND STATE MANAGEMENT

The project uses caching at several levels. On the frontend, data fetching is performed with SWR in route pages. SWR helps reuse fetched API responses and revalidate them when needed. Auth state also uses a small in-memory cache to prevent repeated requests and to keep the navbar responsive. The frontend token cookie enables both client-side API calls and server-side initial auth detection.

At the browser level, media and public storage assets may be cached by the storage provider and browser. On the backend, no large application-level cache was added because the project size is manageable, but the architecture allows future caching for movie lists, trending data, and search results.

A notable backend cache-related issue occurred with `asyncpg` prepared statement caching in Vercel deployment. Since Supabase transaction pooling does not support prepared statements in the same way as direct persistent connections, disabling the `asyncpg` statement cache was required for production stability.

X. HLS STREAMING AND PLAYER FUNCTIONALITY

Video playback is implemented in `frontend/components/Player.tsx`. The platform stores movie stream URLs in the database as `hls_master_url`. These point to public HLS playlist files, usually ending in `.m3u8`. The player uses native browser support where available and `hls.js` where native HLS playback is not available.

A. Playback Features

The player includes:

- play and pause controls;
- 10-second seek forward and backward;
- progress bar with current time and duration;
- volume and mute controls;
- fullscreen toggle;
- subtitles through HTML track elements;
- auto-hide player controls during playback;
- cursor hiding when controls are hidden;
- loading spinner while the stream is preparing;
- stream error handling and retry support.

B. HLS Handling

HTTP Live Streaming uses playlist files and media segments as described in the HLS specification. The browser or `hls.js` reads the playlist and downloads the media segments needed for playback. This architecture is useful for streaming because content can be split into smaller chunks and delivered over standard HTTP infrastructure.

The player checks whether the source is HLS. If it is an HLS source and the browser supports native HLS, the URL is assigned directly to the video element. Otherwise, `hls.js` loads and attaches the stream. The player also includes direct video fallback for non-HLS URLs.

C. Hero Preview

The homepage hero preview is implemented in `frontend/components/HeroPreview.tsx`. It uses muted autoplay playback for the featured movie and loops a short preview window. This provides a more dynamic landing page similar to commercial streaming platforms.

XI. SEARCH FUNCTIONALITY

Search was upgraded from basic text matching to typo-aware ranked retrieval. It is implemented across both frontend and backend.

A. Frontend Search Flow

The search input is located in `frontend/components/SiteHeader.tsx`. When the user types a query, the frontend updates the URL query parameter and routes the user to the homepage search state. The homepage and dedicated search page both call the backend movie API with the query parameter. The frontend also displays result counts, empty states, and ranked results.

Relevant frontend files include:

- `frontend/components/SiteHeader.tsx`;
- `frontend/app/page.tsx`;
- `frontend/app/search/page.tsx`;
- `frontend/lib/api.ts`.

B. Backend Search Ranking

The backend search logic is implemented in `backend/app/api/routes/movies.py`. The search query and movie text are normalized by lowercasing and removing punctuation. Each movie candidate is scored using multiple signals:

- exact title match;
- title prefix match;
- title substring match;
- description substring match;
- token overlap with title and description;
- fuzzy string similarity using `SequenceMatcher`.

The backend computes a relevance score for each candidate, filters weak matches, sorts by score and recency, and returns the best results. This allows the system to return relevant movies even when the user types a partial title or misspells a word. For example, a query such as `spidreman` can still return Spider-Man-related results.

C. Why Fuzzy Search Matters

Exact search can fail easily when users make spelling mistakes or do not remember the full title. A ranked search approach improves usability because it returns the closest available titles instead of showing an empty result too quickly. This makes the platform feel closer to real streaming applications where search is forgiving and relevance-based.

XII. WATCH HISTORY AND CONTINUE WATCHING

Watch history is recorded through the player. When playback reaches a defined progress threshold, the frontend posts progress to the backend. When the video ends, the frontend marks the item as completed. The backend stores progress in the `watch_history` table.

The History page displays this data to users. The homepage also uses history to show a Continue Watching row. This feature helps users resume content and gives the homepage a personalized feel. Although progress resume playback can be extended further, the existing system already stores the required progress seconds for each user/movie pair.

XIII. DEPLOYMENT

Deployment was one of the most important parts of the project because the system had to work outside local development.

A. Frontend Deployment on Vercel

The Next.js frontend is deployed to Vercel. The frontend uses the environment variable `NEXT_PUBLIC_API_BASE` to point to the deployed backend API. The production frontend is served over HTTPS and communicates with the backend through REST API calls.

B. Backend Deployment on Vercel

The FastAPI backend is also deployed to Vercel. Vercel supports FastAPI applications when a FastAPI app instance is exposed through a valid entry point. The backend includes a `server.py` entry and a `vercel.json` configuration. The backend is deployed as a serverless function, which means database connection handling must be compatible with short-lived execution environments.

C. Environment Variables

Production environment variables include database URLs, auth secret keys, CORS origins, SSL verification behavior, and API base URLs. Local development uses local `.env` files, while Vercel environment variables are configured in the Vercel dashboard. This separation avoids hardcoding secrets and keeps local and production environments configurable.

D. Supabase Pooler Configuration

In production, the backend uses the Supabase transaction pooler through `DATABASE_POOL_URL`. This is better suited to serverless execution than direct database connections. The backend also uses `NullPool` in serverless mode to avoid keeping idle connections inside short-lived functions.

XIV. CORS POLICY AND CROSS-ORIGIN AUTHENTICATION

Because the frontend and backend are deployed on different domains, CORS configuration is necessary. The backend uses FastAPI's `CORSMiddleware` and a custom fallback middleware. The policy allows configured origins, supports credentials, handles preflight `OPTIONS` requests, and adds CORS headers even when an internal server error occurs.

The production configuration includes the frontend Vercel origin. A regex fallback can also allow Vercel preview domains. This was important because earlier failures could make browser requests appear as CORS errors even when the underlying issue was a backend exception. Adding consistent CORS headers helped distinguish real backend errors from browser CORS blocking.

Cross-origin authentication was handled carefully. HTTP-only backend cookies are secure, but cookies set by the backend domain are not directly readable by the frontend domain. To keep server-rendered and client-rendered auth state consistent, the frontend stores a token in its own domain and sends it as a Bearer token to the backend. The backend accepts both cookie and Bearer-token authentication.

XV. ERROR HANDLING AND PRODUCTION DEBUGGING

The project included several real debugging tasks that improved production reliability.

A. Internal Server Error Debugging

The backend originally returned generic internal server errors in deployment. Stack trace logging was added so that the actual production cause could be found from Vercel runtime logs. This made it possible to identify database connection and prepared statement issues.

B. Prepared Statement Conflict

One major issue was `DuplicatePreparedStatementError` from `asyncpg` when using the Supabase transaction pooler. This occurred because transaction pooling and `asyncpg` statement caching were incompatible in the deployed serverless context. The solution was to disable `asyncpg` statement caching in Vercel by setting `statement_cache_size` to zero.

C. Graceful API Fallbacks

Some APIs were made defensive. For example, if optional genre/subtitle hydration fails, the movie can still be returned. If history access fails, safe empty or fallback responses prevent the frontend from crashing. These decisions improve user experience and make the platform more resilient during partial data or migration issues.

D. Frontend Error and Empty States

The frontend includes better empty states for search and history, better loading skeletons, and user-friendly error panels. This improves the polish of the application and makes it easier to understand system behavior during loading, no-data, or failure states.

XVI. FEATURE SUMMARY

Table III summarizes the major implemented features.

XVII. SOFTWARE ENGINEERING PRINCIPLES APPLIED

The project followed modern software engineering practices commonly used in industry-grade web development.

A. Separation of Concerns

Frontend rendering, backend business logic, database models, authentication utilities, and API helpers are separated into different files and modules. This improves maintainability and makes it easier to assign work among team members.

B. Reusable Components

Components such as `MovieCard`, `Row`, `Player`, `SiteHeader`, `AuthNav`, and `Skeletons` are reusable. This avoids duplicate UI code and makes future changes easier.

C. Environment-Based Configuration

The app uses environment variables for API base URLs, database URLs, secret keys, CORS origins, and SSL behavior. This prevents hardcoding environment-specific values and makes local and production deployments easier to manage.

D. Defensive Programming

The backend includes graceful fallbacks for optional data failures, CORS fallback headers during errors, and explicit handling for DB connection behavior. The frontend includes error panels, loading states, and retry actions.

E. Incremental Production Debugging

Instead of assuming local behavior would match deployment behavior, the team inspected runtime logs, reproduced errors, and applied targeted fixes. Examples include handling CORS, serverless database pooling, statement cache conflicts, auth cookie behavior, and player loading states.

XVIII. TESTING AND VALIDATION

The system was tested through local development, API calls, frontend builds, and deployment testing. The frontend was validated using `npm run build`. Backend endpoints were tested with local Uvicorn runs and curl requests. Production backend behavior was validated through Vercel logs and deployed API endpoints.

Tested areas include:

- user registration and login;
- current-user authentication checks;
- protected route redirects;
- movie listing and detail retrieval;
- search with exact, partial, and typo queries;
- history updates from player activity;
- deployed CORS preflight and actual requests;
- serverless database connection behavior;
- frontend production build compilation.

XIX. LIMITATIONS

The current system is functional but still has limitations. The recommendation logic is simple and not machine-learning based. The search ranking is based on string similarity rather than semantic embeddings. Admin upload and content management are not fully implemented as a complete dashboard. Resume playback can be extended so that the player automatically starts from the last saved progress. HLS stream correctness also depends on the media playlist and segment files being valid and publicly accessible in storage.

XX. FUTURE WORK

Future improvements could include:

- admin dashboard for uploading movies, posters, subtitles, and HLS files;
- automatic HLS transcoding pipeline;
- semantic search using embeddings;
- stronger recommendation engine using user behavior and genres;
- resume playback from last watched position;
- user watchlist and favorites;
- role-based access control for admins;
- analytics dashboard for search, viewing, and errors;
- automated tests for backend routes and frontend components.

TABLE III
IMPLEMENTED FEATURE SUMMARY

Category	Feature	Implementation Summary
Authentication	Register, login, logout, current user	FastAPI routes, JWT, bcrypt password hashing, cookie and Bearer token support.
Route Security	Protected pages	Client RouteGuard checks auth and redirects logged-out users.
Navigation	Navbar auth and search	Server-seeded auth state, stable auth nav, visible search.
Homepage	Hero, rows, search state, continue watching	Homepage fetches movie/history data and renders responsive UI.
Movie Details	More-info pages	Dedicated pages show metadata and play actions.
Playback	HLS player	HTML5 video, hls.js, controls, seek, volume, fullscreen, subtitles, spinner.
History	Watch progress	Player posts progress; backend upserts watch history.
Search	Typo-aware ranked search	Normalization, exact/partial/token/fuzzy scoring in backend.
Database	PostgreSQL models	Users, movies, genres, movie genres, subtitles, watch history.
Deployment	Vercel + Supabase	Frontend and backend deployed on Vercel; DB and storage on Supabase.
CORS	Cross-origin API access	Explicit allowed origins, preflight handling, credential support.
Production Stability	Serverless DB fixes	Pooler URL, NullPool, SSL config, asyncpg statement cache disabled.
UI Polish	Loading and empty states	Shared shimmer skeletons, error panels, focus states, responsive design.

XXI. DETAILED IMPLEMENTATION ARTIFACTS

The project is organized so that each major feature has a clear implementation location. Table IV lists representative files and their purpose.

XXII. MAIN USER JOURNEY

A normal user begins at the login or registration page. After creating an account or signing in, the backend issues a JWT and the frontend updates its auth state. The route guard then allows the user to access protected pages. On the homepage, the user can browse the hero feature, latest movies, trending content, and Continue Watching items. If the user enters text in the search bar, the query is reflected in the route and sent to the backend movie API. The backend ranks candidate movies and returns relevant results.

When the user opens a movie details page, the frontend fetches metadata from the backend and displays the available information. From there, the user can start playback. During playback, the player loads the HLS playlist or direct stream URL, attaches it to the video element, and provides controls for play, pause, seek, volume, mute, and fullscreen. The player records progress by posting to the history API. Later, the history page and Continue Watching row use this stored progress to personalize the experience.

This journey shows how auth state, API data, database records, media URLs, and UI state work together.

XXIII. API ENDPOINT SUMMARY

The backend API is intentionally small but complete enough to support the main streaming workflow. Table V summarizes the important endpoints.

The API uses JSON bodies, Pydantic schemas for external contracts, and SQLAlchemy models for database representation.

XXIV. SECURITY CONSIDERATIONS

The project applies practical security measures. Passwords are hashed before database insertion, JWTs include expiration, and protected backend routes use a shared current-user dependency. CORS allows expected frontend origins and Vercel preview origins instead of blindly allowing every origin with credentials. Cookie settings differ between local HTTP and production HTTPS, and Bearer tokens are supported because frontend and backend run on separate domains. Secrets such as database URLs and JWT keys are stored in environment variables rather than source code.

XXV. CHALLENGES FACED AND SOLUTIONS

Several practical issues occurred during development and deployment. The deployed backend initially returned internal server errors even though local execution worked. Vercel logs showed an asyncpg and Supabase transaction-pooler conflict, which was fixed by disabling prepared statement caching in serverless deployment. CORS also required careful handling, so fallback CORS headers were added even for error responses. Cross-domain authentication was stabilized by combining backend cookie auth with a frontend-domain token cookie and Bearer header fallback. UI loading states were refined with server-seeded auth state, skeleton loaders, and cleaner empty/error panels. Video playback was improved through hls.js, native playback support, and explicit loading/error states.

XXVI. TEAM CONTRIBUTION MODEL

The project can be divided into four ownership areas: frontend UI/UX, authentication and user flow, backend/database/deployment integration, and search functionality. This division helps each member present one major subsystem while still understanding how it connects to the full application.

TABLE IV
IMPORTANT IMPLEMENTATION FILES

File	Responsibility
frontend/app/layout.tsx	Global layout, shared header, server-seeded auth, and route guard wrapper.
frontend/components/SiteHeader.tsx	Navbar, search input, mobile menu, player-page chrome hiding, and route-aware search visibility.
frontend/components/AuthNav.tsx	Login/logout buttons, history link, auth cache synchronization, and logout behavior.
frontend/components/RouteGuard.tsx	Client-side protected route check and redirect to login for unauthenticated users.
frontend/app/page.tsx	Homepage data fetching, hero section, movie rows, search result state, result count, and Continue Watching row.
frontend/app/search/page.tsx	Dedicated search page, ranked results display, empty state, and result count.
frontend/components/Player.tsx	Main video player, HLS/direct video loading, playback controls, buffering state, history posting, and fullscreen.
frontend/lib/api.ts	Central frontend API helper, base URL configuration, credentials, and Authorization header.
frontend/lib/auth.ts	Frontend token storage, current-user fetching, auth cache invalidation, and shared auth helper.
frontend/lib/server-auth.ts	Server-side auth detection for initial navbar user state.
backend/app/main.py	FastAPI app creation, router registration, health endpoint, CORS, and fallback error/CORS middleware.
backend/app/api/routes/auth.py	Register, login, logout, and current-user API endpoints.
backend/app/api/routes/movies.py	Movie list, details, trending, latest releases, recommendations, hydration, and fuzzy search.
backend/app/api/routes/history.py	Watch-history list and upsert APIs.
backend/app/db/session.py	Async database engine, SSL behavior, Supabase pooler URL selection, and serverless-safe connection configuration.
backend/app/core/security.py	JWT generation, password hashing, password verification, and bcrypt compatibility.

TABLE V
CORE API ENDPOINTS

Endpoint	Purpose
POST/api/v1/auth/register	Create a new user account.
POST/api/v1/auth/login	Verify credentials, issue JWT, and set auth cookie.
POST/api/v1/auth/logout	Clear auth cookie and end frontend session.
GET/api/v1/auth/me	Return current authenticated user.
GET/api/v1/movies	List movies or perform query-based fuzzy search.
GET/api/v1/movies/trending	Return trending movies with fallback.
GET/api/v1/movies/latest	Return recent movies.
GET/api/v1/movies/{id}	Return one movie with related data.
GET/api/v1/movies/{id}/recommendations	Return related movies.
GET/api/v1/history	Return user watch history.
POST/api/v1/history	Upsert watch progress.
GET/health	Backend health check.

XXVII. CONCLUSION

This project demonstrates a cloud-based movie streaming platform with a Next.js frontend, FastAPI backend, PostgreSQL/Supabase database, HLS playback, authentication, watch history, typo-aware search, UI polish, CORS handling, and Vercel deployment. The work also addressed practical production issues such as cross-origin auth, serverless database pooling, prepared-statement conflicts, loading states, and deployment configuration. Overall, the system shows how a modern streaming application can combine frontend UX, backend APIs, database modeling, media delivery, and cloud deployment into one integrated product.

CREDITS AND USE OF TOOLS

The project architecture, feature planning, integration decisions, testing, and final ownership remained with the team. ChatGPT and Codex were used as AI-assisted tools for selected implementation help, debugging, grammar/report polishing, and code refinement. The team used modern software engineering practices and reviewed, adapted, and integrated all final work.